

Report of Assignment – 3

Course: MA6106 (LLM)



Indian Institute of Technology Patna

Submitted to:

Dr Ranjeet Ranjan Jha

Department of Mathematics

Indian Institute of Technology Patna

Submitted by: *Group 3*

1. Antra Sinha (2621MA06)
2. Saurav Kumar (2611MC08)
3. Ratnesh Pandey (2611MC19)

Table of Contents

1. Overview 3

2. Objectives 3

3. Dataset Generation and Preprocessing..... 3

 3.1 Gaussian Dataset 3

 3.2 Spiral Dataset 3

 3.3 Data Split and Standardization 4

4. FCNN Methodology 4

 4.1 Network Structure 4

 4.2 Forward Propagation 4

 4.3 Loss Function 4

 4.4 Backpropagation and SGD 4

5. Architecture Experiments 5

 5.1 Gaussian Architectures..... 5

 5.2 Spiral Architectures 5

6. Best-Model Validation and Test Results 5

7. Decision Regions 9

8. Internal Node-Output Visualizations 10

9. Single-Neuron Baseline 12

11. Conclusion 12

TASK 2: 12

1. Objectives 12

2. Dataset and Preprocessing 12

 2.1 Fashion-MNIST Dataset..... 12

 2.2 Data Split..... 13

 2.3 Input Representation..... 13

3. Methodology 14

 3.1 Fully Connected Neural Network Design 14

 3.2 Loss Function 14

 3.3 Optimization Methods 14

 3.4 Initialization and Convergence..... 14

4. Experimental Results and Analysis 14

5. Conclusion 18

Overview

This report presents a combined analysis of two neural-network experiments. Task 1 studies fully connected neural networks (FCNNs) on two synthetic three-class datasets—Gaussian clusters and a nonlinear spiral dataset—using a from-scratch implementation with sigmoid activations, squared error, backpropagation and stochastic gradient updates. Task 2 extends the study to five classes of Fashion-MNIST and evaluates three FCNN depths with six optimization methods: SGD, Batch GD, Momentum, NAG, RMSProp and Adam. The results show that dataset geometry strongly affects the required model capacity, while optimization choice has a major effect on convergence and final accuracy. Task 1 achieves 100.00% test accuracy on the Gaussian data and 98.67% on the spiral data with the selected architectures. Task 2 achieves 91.70% test accuracy with a four-hidden-layer network trained using SGD.

2. Objectives

- Generate balanced three-class Gaussian and spiral datasets.
- Apply a stratified 60/20/20 train-validation-test split.
- Standardize the input features using statistics computed from the training set.
- Implement an FCNN from scratch with sigmoid activation and squared-error loss.
- Compare several hidden-layer architectures for each dataset.
- Select the best architecture using validation accuracy.
- Evaluate the selected models on the test set using accuracy and confusion matrices.
- Visualize learned decision regions and internal node outputs.
- Compare the FCNN against a single-neuron softmax baseline.

3. Dataset Generation and Preprocessing

3.1 Gaussian Dataset

The Gaussian dataset contains 500 samples per class, giving 1,500 observations in total. The three classes are generated from different two-dimensional Gaussian distributions with distinct means and covariance matrices. The resulting clusters are visually separated.

3.2 Spiral Dataset

The spiral dataset also contains 500 samples per class. Each class forms a curved spiral arm, with radial position increasing from the center and Gaussian noise added to the angular coordinate. This construction creates a strongly nonlinear classification boundary.

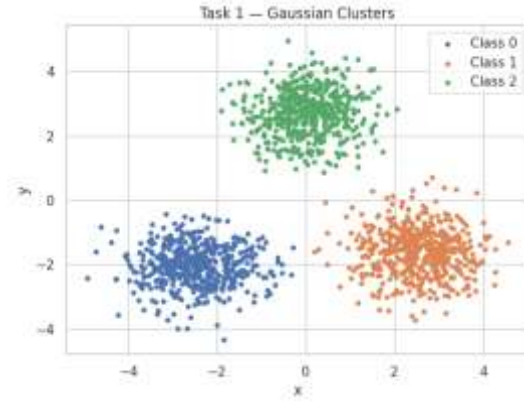


Figure 1. Generated Gaussian clusters and spiral dataset.

3.3 Data Split and Standardization

For both datasets, 60% of the samples are used for training, 20% for validation, and 20% for testing. Because there are 1,500 samples in each dataset, the resulting split sizes are 900 training samples, 300 validation samples, and 300 test samples. The split is stratified so that each subset preserves the three-class distribution. Standardization is performed using the training-set mean and standard deviation, which are then applied to the validation and test sets.

Dataset	Training	Validation	Test	Total
Gaussian	900	300	300	1500
Spiral	900	300	300	1500

4. FCNN Methodology

4.1 Network Structure

The FCNN accepts two input features and produces three output values corresponding to the three classes. For the Gaussian dataset, one hidden layer is used. For the spiral dataset, two hidden layers are used. Every hidden and output unit applies the sigmoid activation function.

4.2 Forward Propagation

For a layer, the pre-activation is computed as $z = aW + b$ and the activation is $a = \text{sigmoid}(z)$. The output of one layer becomes the input to the next layer. During prediction, the class with the largest output activation is selected.

4.3 Loss Function

The training objective is average squared error between the one-hot encoded target vector and the network output. The implementation uses the form $0.5 \times \sum (y - \hat{y})^2$ for each sample before averaging over the dataset.

4.4 Backpropagation and SGD

Gradients are propagated from the output layer toward the input layer using the derivative of the sigmoid activation. Parameters are updated after every training example with learning rate 0.03. Training uses shuffled samples and stops when the change in training error becomes smaller than $1e-5$, subject to the maximum epoch limit.

5. Architecture Experiments

5.1 Gaussian Architectures

Architecture	Hidden Nodes	Epochs	Train Acc.	Validation Acc.
(2, 3, 3)	(3,)	147	100.00%	100.00%
(2, 5, 3)	(5,)	114	100.00%	100.00%
(2, 8, 3)	(8,)	103	100.00%	100.00%
(2, 12, 3)	(12,)	102	100.00%	100.00%
(2, 16, 3)	(16,)	97	100.00%	100.00%

All five Gaussian architectures achieve perfect training and validation accuracy. Increasing the hidden-layer width mainly changes the number of epochs needed to reach convergence; it does not improve the final classification accuracy because the classes are already well separated.

5.2 Spiral Architectures

Architecture	Hidden Nodes	Epochs	Train Acc.	Validation Acc.
(2, 5, 5, 3)	(5, 5)	149	98.00%	98.33%
(2, 8, 5, 3)	(8, 5)	129	98.22%	99.00%
(2, 10, 8, 3)	(10, 8)	36	56.89%	58.00%
(2, 15, 10, 3)	(15, 10)	73	84.33%	84.67%
(2, 20, 12, 3)	(20, 12)	156	99.56%	99.00%

The spiral task shows a much larger dependence on architecture. The architecture (2, 8, 5, 3) obtains the highest validation accuracy of 99.0%. The (2, 20, 12, 3) network also performs strongly, whereas (2, 10, 8, 3) converges quickly but reaches only 58.0% validation accuracy. This shows that convergence alone is not sufficient; the learned decision function must also be appropriate for the data.

6. Best-Model Validation and Test Results

The best Gaussian model is (2, 3, 3), selected by validation accuracy. The best spiral model is (2, 8, 5, 3). Both selected models are then evaluated on the held-out test sets.

Dataset	Best Architecture	Validation Accuracy	Test Accuracy
Gaussian	(2, 3, 3)	100.00%	100.00%
Spiral	(2, 8, 5, 3)	99.00%	98.67%

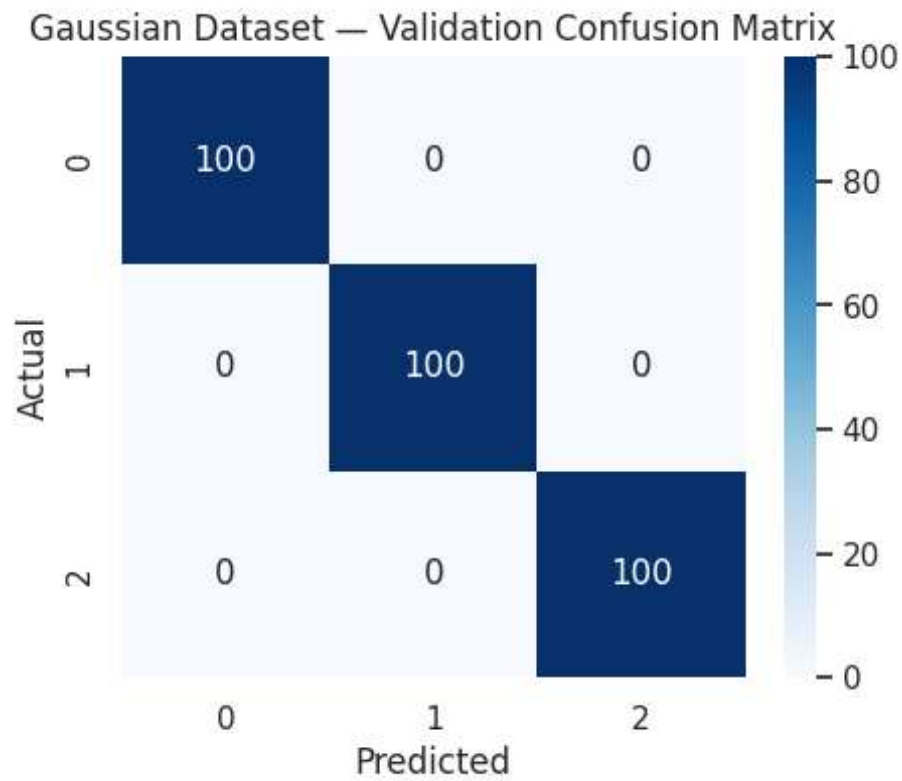


Figure 2. Gaussian validation confusion matrix for the best architecture.

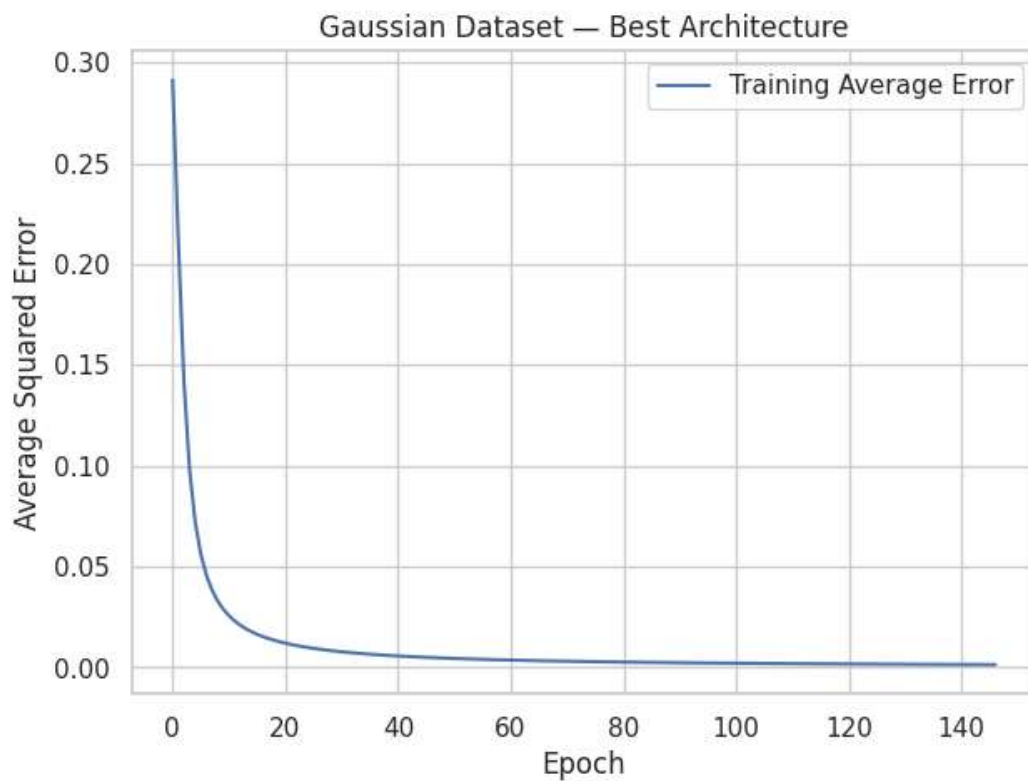


Figure 3. Gaussian training average squared error.

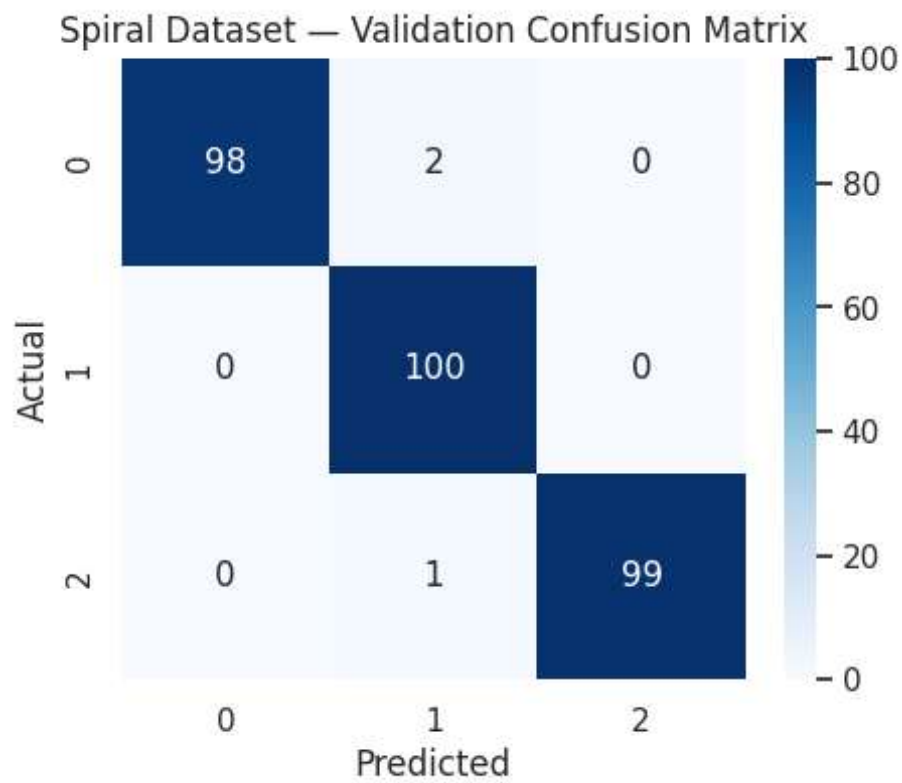


Figure 4. Spiral validation confusion matrix for the best architecture.

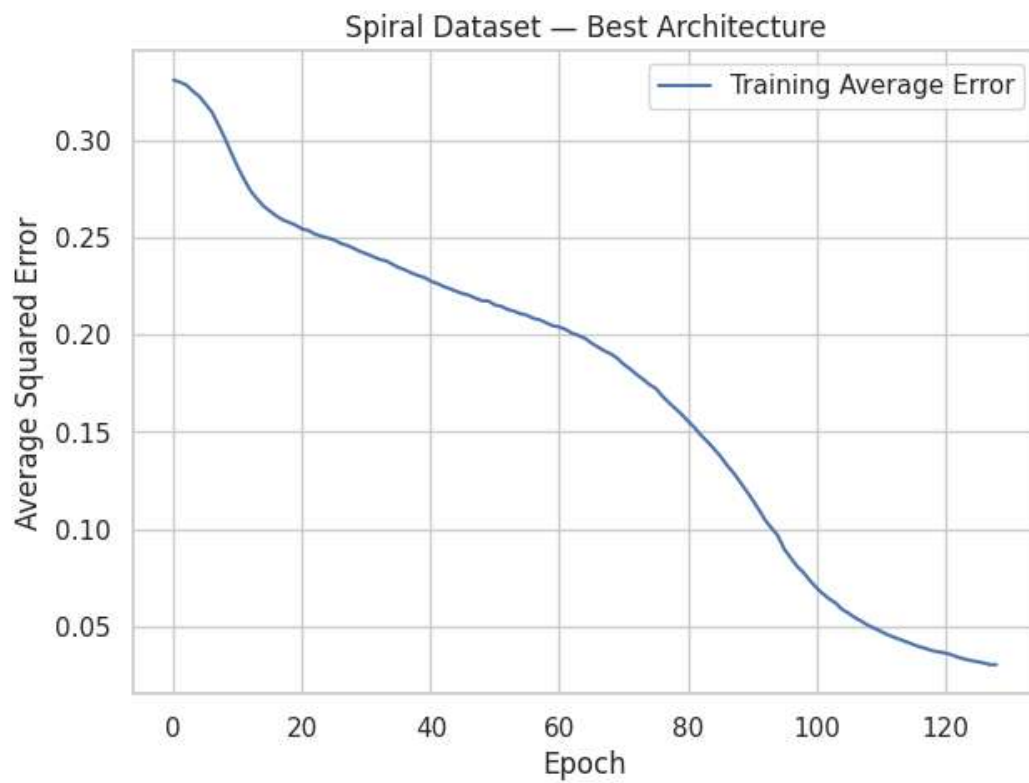


Figure 5. Spiral training average squared error.

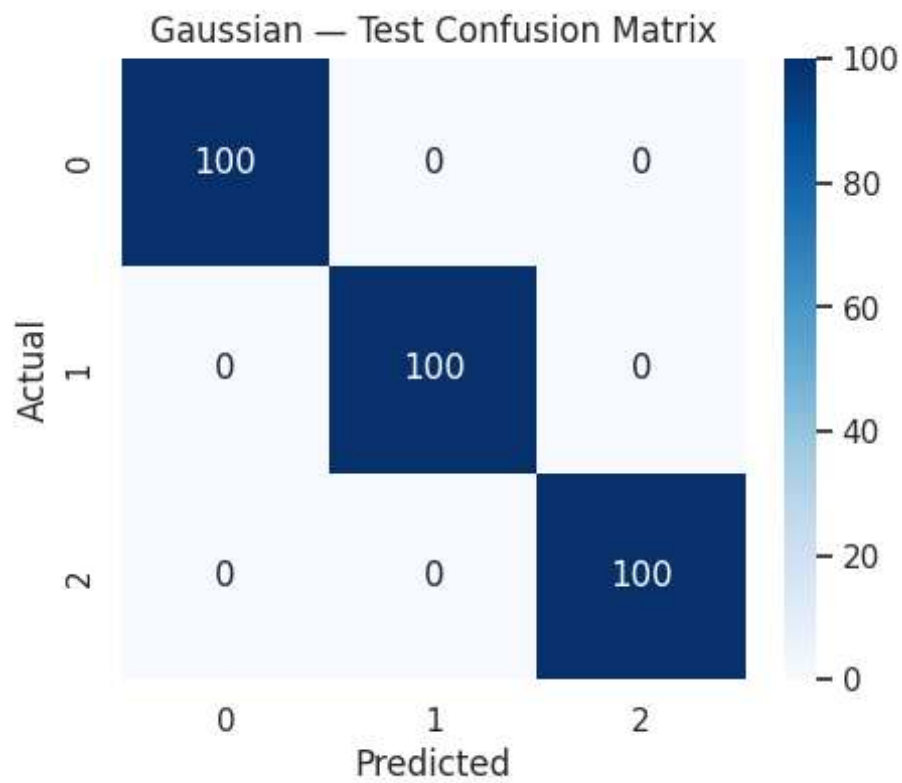


Figure 6. Gaussian test confusion matrix.

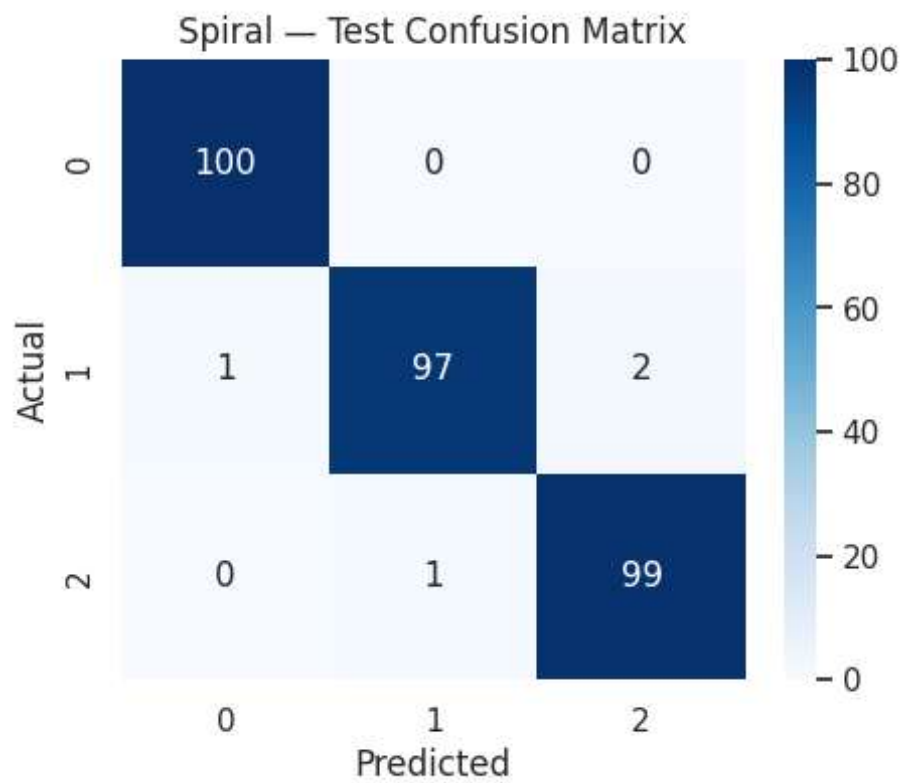


Figure 7. Spiral test confusion matrix.

The confusion matrices confirm the numerical accuracy results. The Gaussian model classifies every validation and test example correctly. The spiral model makes only a small number of errors: on the test set, the confusion matrix contains 100 correct predictions for class 0, 97 correct predictions for class 1, and 99 correct predictions for class 2, with four misclassified samples in total.

7. Decision Regions

Decision-region plots provide a geometric view of what the trained FCNN has learned. The Gaussian model produces simple, clean regions that align with the three compact clusters. In contrast, the spiral model learns curved regions that follow the intertwined spiral structure. This is direct visual evidence that hidden layers allow the FCNN to construct nonlinear class boundaries.

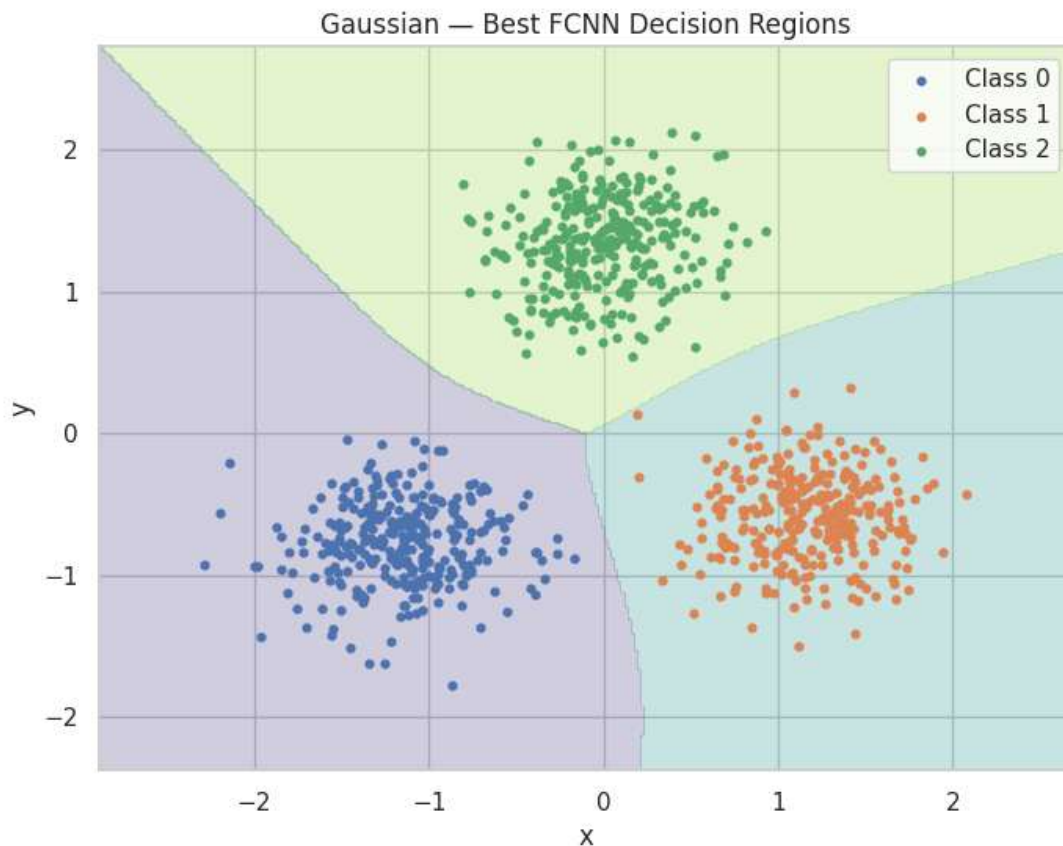


Figure 8. Decision regions learned by the best Gaussian FCNN.

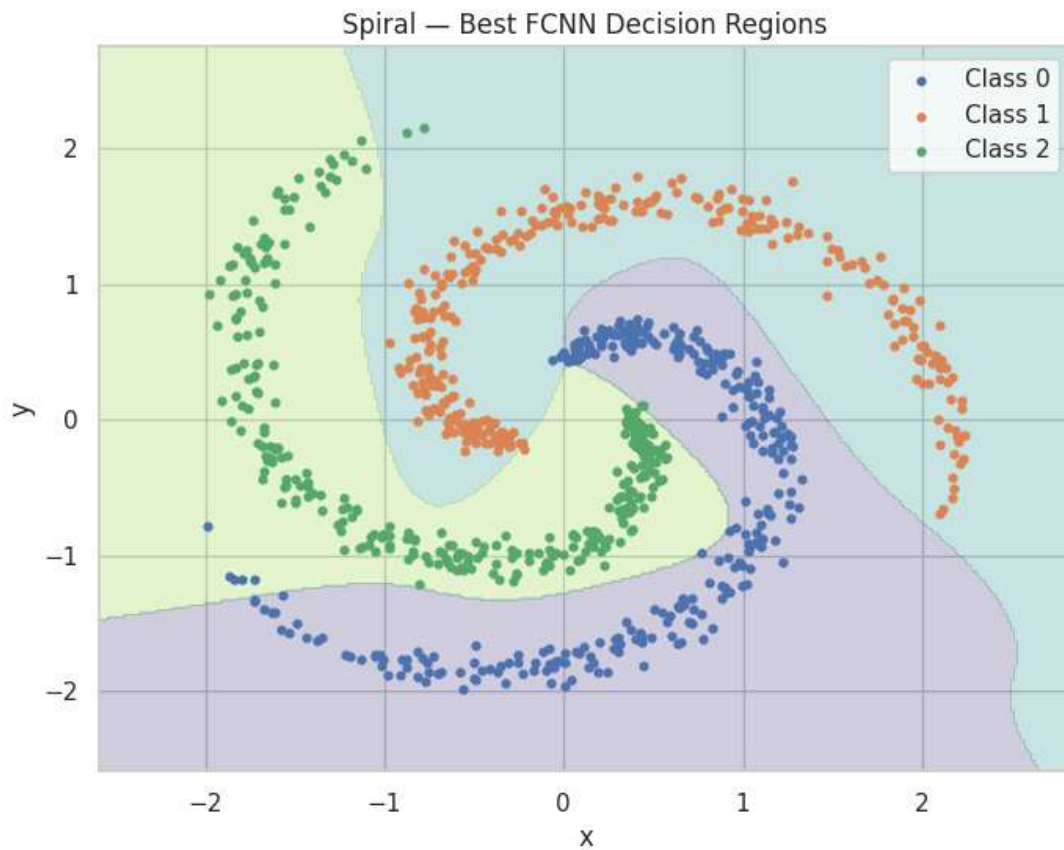


Figure 9. Decision regions learned by the best spiral FCNN.

8. Internal Node-Output Visualizations

The notebook also plots the output of every hidden and output node in three dimensions against the original two input features. These plots illustrate how the network transforms the original input space through successive layers. For the Gaussian problem, individual node responses form clearly separated structures. For the spiral problem, internal activations progressively encode the nonlinear geometry needed for classification.

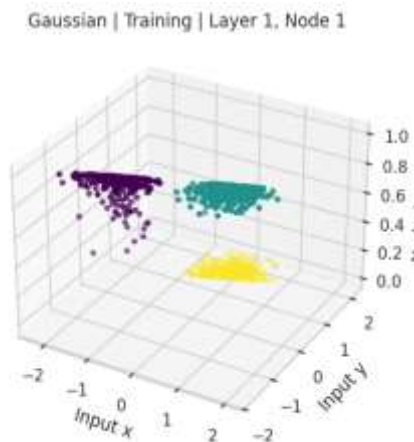


Figure 10. Representative Gaussian hidden-node output: training set, Layer 1 Node 1.

Gaussian | Training | Layer 1, Node 3

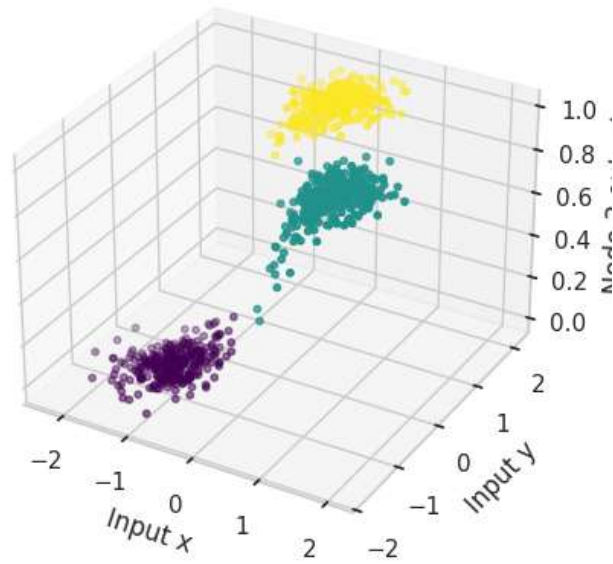


Figure 11. Representative Gaussian hidden-node output: training set, Layer 1 Node 3.

Spiral | Training | Layer 1, Node 1

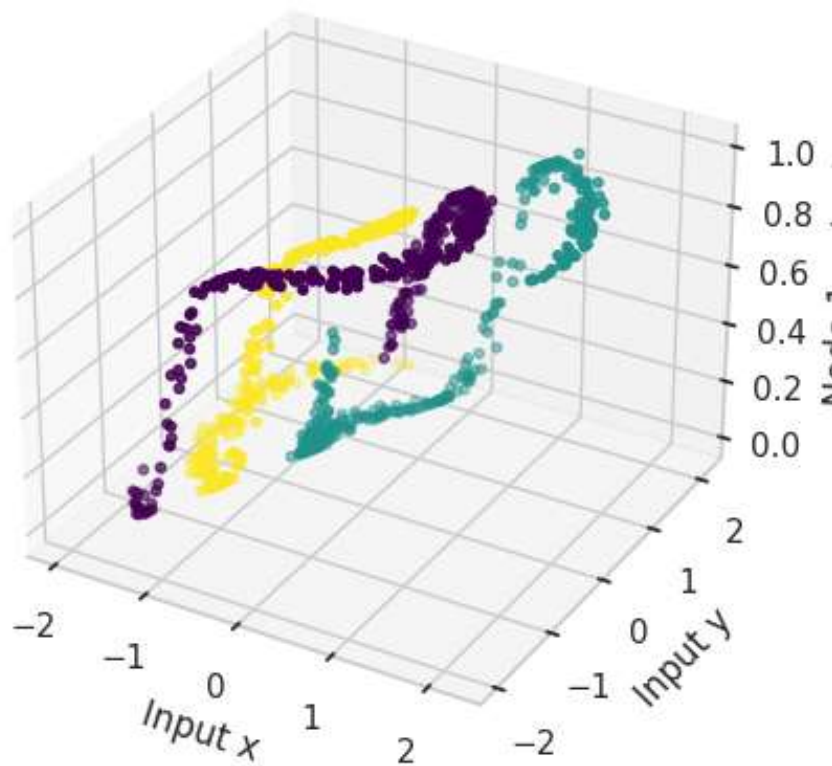


Figure 12. Representative spiral hidden-node output.

9. Single-Neuron Baseline

A single-neuron softmax classifier is included as a simple linear baseline. It performs perfectly on the Gaussian dataset because those classes are linearly separable enough for this setting. On the spiral dataset, however, the baseline reaches only 51.33% test accuracy, confirming that a linear decision rule is inadequate for the nonlinear spiral structure.

Dataset	Training Accuracy	Validation Accuracy	Test Accuracy
Gaussian	100.00%	100.00%	100.00%
Spiral	51.78%	52.33%	51.33%

11. Conclusion

The experiments demonstrate the usefulness of fully connected neural networks for multiclass classification and highlight the importance of architecture when the underlying decision boundary is nonlinear. The Gaussian dataset can be solved by very small networks, while the spiral dataset benefits from a carefully chosen two-hidden-layer model. The best spiral FCNN achieves 99.0% validation accuracy and 98.67% test accuracy, substantially outperforming the single-neuron baseline. Overall, the results show that FCNNs trained with backpropagation and SGD can learn both simple and highly nonlinear two-dimensional class structures when the network architecture is appropriately selected.

TASK 2:

1. Objectives

- Implement an FCNN for five-class Fashion-MNIST classification.
- Create a balanced dataset containing the five selected categories.
- Compare FCNNs with three, four, and five hidden layers.
- Evaluate SGD, Batch Gradient Descent, Momentum, NAG, RMSProp, and Adam.
- Examine the convergence behavior of the different optimizers.
- Compare training and validation accuracy across all architecture–optimizer pairs.
- Select the strongest model using validation accuracy.
- Assess the selected model on unseen test data using accuracy and confusion matrices.

2. Dataset and Preprocessing

2.1 Fashion-MNIST Dataset

The Fashion-MNIST data used in the experiment was loaded through the torchvision implementation. The original dataset consists of 60,000 training images and 10,000 test images, with each image represented as a 28×28 grayscale image. Five categories were selected for the present study. Each selected category contributes 7,000 samples, giving a balanced dataset of 35,000 images.

Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat

2.2 Data Split

A stratified partition was applied so that the class proportions remained identical across the training, validation, and test subsets. The final split contained 22,400 training images, 5,600 validation images, and 7,000 test images. Every subset remained perfectly balanced across the five classes.

Subset	Samples	Percentage
Training	22,400	64%
Validation	5,600	16%
Test	7,000	20%
Total	35,000	100%

Class	Training	Validation	Test
T-shirt/top	4480	1120	1400
Trouser	4480	1120	1400
Pullover	4480	1120	1400
Dress	4480	1120	1400
Coat	4480	1120	1400

2.3 Input Representation

Each image contains $28 \times 28 = 784$ pixels. Consequently, the image is flattened into a 784-dimensional vector before being supplied to the FCNN. Pixel intensities are normalized to the interval $[0, 1]$, producing an input representation of $35,000 \times 784$ for the complete selected dataset.

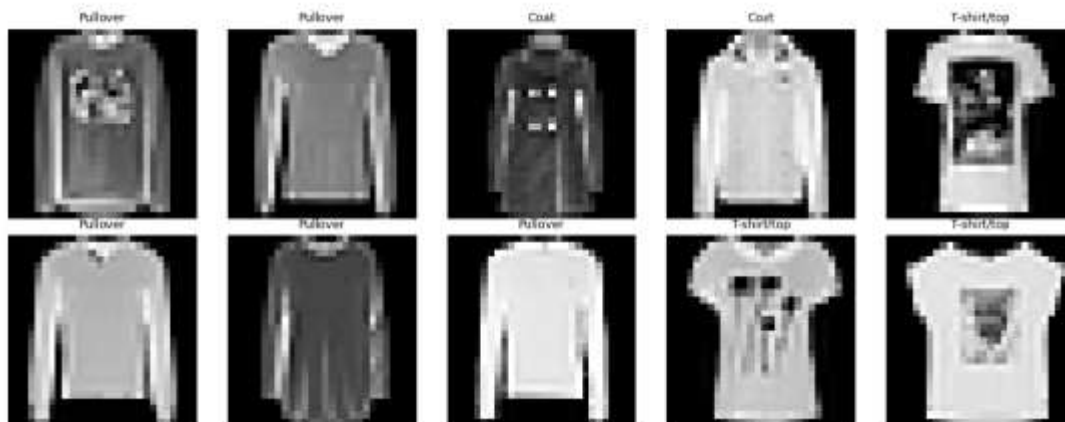


Figure 1. Representative training samples from the selected Fashion-MNIST classes.

3. Methodology

3.1 Fully Connected Neural Network Design

The classifier uses a fully connected architecture with 784 input neurons, reflecting the flattened image representation. ReLU activation is applied in the hidden layers, while the final layer contains five output neurons, one for each target class. Three network depths were investigated.

Architecture	Hidden-layer structure	Parameters
3-hidden	$128 \rightarrow 64 \rightarrow 32$	110,981
4-hidden	$128 \rightarrow 64 \rightarrow 32 \rightarrow 16$	111,429
5-hidden	$128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8$	111,525

The architecture that ultimately performed best had the structure $784 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 5$. Each hidden layer computes a linear transformation followed by ReLU activation, while the output layer produces class scores.

3.2 Loss Function

Cross-entropy was used as the objective function. In the implementation, this loss was realized through PyTorch's `CrossEntropyLoss`, with the target labels compared against the predicted class probabilities.

3.3 Optimization Methods

Six optimization approaches were compared under the experimental settings reported in the original notebook/report:

Optimizer	Configuration
SGD	Learning rate $\alpha = 0.001$; batch size = 1
Batch Gradient Descent	Entire training set used for each update
Momentum	$\alpha = 0.001$; $\mu = 0.9$
NAG	$\alpha = 0.001$; $\mu = 0.9$
RMSProp	$\alpha = 0.001$; $\beta = 0.99$; $\epsilon = 10^{-8}$
Adam	$\alpha = 0.001$; $\beta_1 = 0.9$; $\beta_2 = 0.999$; $\epsilon = 10^{-8}$

3.4 Initialization and Convergence

A random seed of 42 was fixed for reproducibility. For each architecture, the same saved initial weights were reused when comparing optimizers, reducing the effect of different starting points. Training was allowed to continue for up to 100 epochs. Convergence was declared when the absolute change in average training error between consecutive epochs fell below 0.001, with convergence checks beginning after the fifth epoch. This design produced $3 \times 6 = 18$ experiments.

4. Experimental Results and Analysis

4.1 Overall Training and Validation Performance

The 18 experiments produced noticeably different results. The four-hidden-layer SGD model ranked first, with 95.19% training accuracy and 91.41% validation accuracy after 30 epochs. The five-hidden-layer SGD model was a close second, while the three-hidden-layer SGD model achieved slightly higher training accuracy but lower validation accuracy. At the opposite end, Batch Gradient Descent remained close to the 20% chance level expected for five balanced classes.

Architecture	Optimizer	Epochs	Train Acc.	Val. Acc.
4-hidden	SGD	30	95.19%	91.41%
5-hidden	SGD	29	94.99%	91.20%
4-hidden	NAG	22	93.47%	91.09%
3-hidden	SGD	40	95.30%	90.80%
4-hidden	Momentum	13	92.00%	90.57%
5-hidden	Momentum	30	92.61%	90.41%
3-hidden	Adam	35	92.81%	90.41%
3-hidden	Momentum	32	94.09%	90.36%
4-hidden	Adam	14	91.49%	89.80%
5-hidden	Adam	29	91.22%	89.59%
3-hidden	NAG	15	90.64%	88.96%
5-hidden	RMSProp	8	87.98%	87.57%
5-hidden	NAG	36	88.77%	86.77%
4-hidden	RMSProp	10	86.40%	85.96%
3-hidden	RMSProp	100	55.86%	55.41%
3-hidden	Batch GD	6	21.10%	20.66%
4-hidden	Batch GD	6	19.99%	20.00%
5-hidden	Batch GD	6	20.00%	20.00%

4.2 Convergence Behavior

Architecture	SGD	Batch GD	Momentum	NAG	RMSProp	Adam
3-hidden	40	6	32	15	100	35
4-hidden	30	6	13	22	10	14
5-hidden	29	6	30	36	8	29

The convergence counts reveal that numerical convergence and useful learning are not the same thing. Batch Gradient Descent reached the stopping criterion in only six epochs, but its accuracy stayed around chance level. RMSProp converged quickly for some architectures but was unstable for the three-hidden-layer model, which reached the maximum of 100 epochs without meeting the criterion.

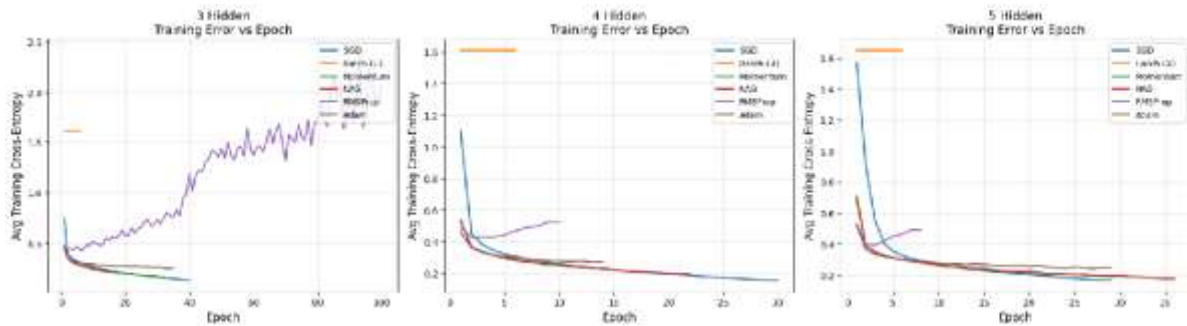


Figure 2. Training-error curves for the tested architecture-optimizer combinations.

4.3 Interpretation of Training Curves

The training-error plots indicate that SGD produced a steady reduction in error across all three architectures. Momentum and NAG also showed effective optimization behavior. Adam reduced error relatively rapidly during the early part of training, but its final validation accuracy was below that of SGD. RMSProp was particularly unstable for the three-hidden-layer network, while Batch Gradient Descent showed little evidence of useful learning under the selected hyperparameters.

4.4 Best Architecture and Optimizer

The highest validation accuracy was obtained by the four-hidden-layer FCNN trained with SGD. Its training accuracy was 95.19%, its validation accuracy was 91.41%, and it converged in 30 epochs. Therefore, the selected model for final testing was the four-hidden-layer FCNN with SGD.

4.5 Test-Set Evaluation

After model selection based on validation performance, the chosen network was evaluated on the unseen test set. It achieved 91.70% test accuracy. The close relationship between validation and test performance suggests that the selected model transferred its learned decision boundaries reasonably well to previously unseen images.

Metric	Accuracy
Training	95.19%
Validation	91.41%
Test	91.70%

4.6 Confusion-Matrix Analysis

The confusion matrices provide a class-by-class view of the model's predictions. They make it possible to identify which clothing categories are recognized reliably and which categories are more frequently confused. The supplied report presents separate matrices for the training and test sets.

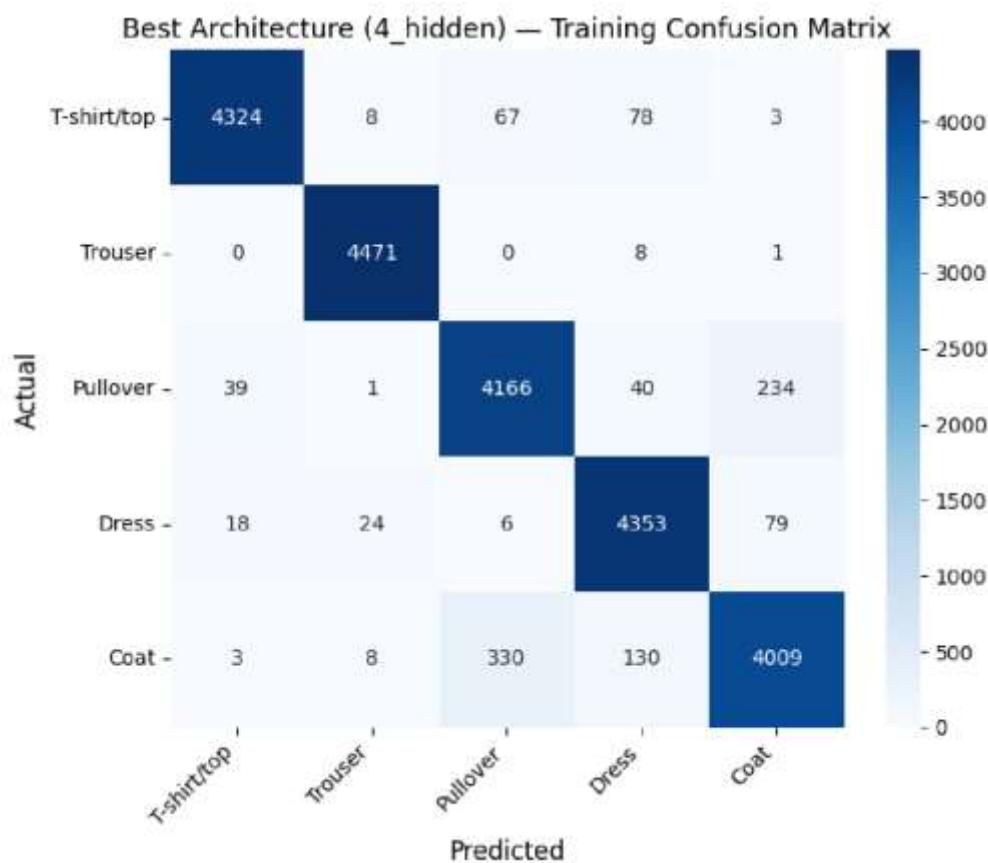


Figure 3. Confusion matrix for the training set.

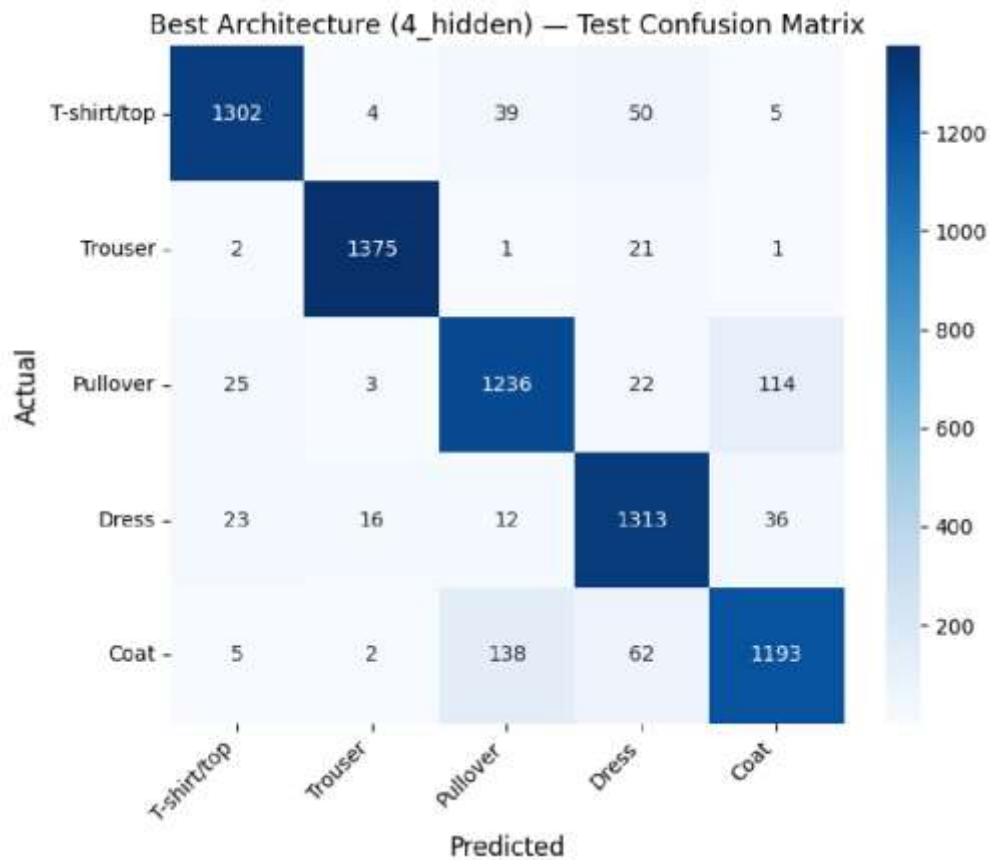


Figure 4. Confusion matrix for the test set.

4.7 Comparison of Network Depth

When SGD is held constant, the four-hidden-layer model provides the best validation result. Moving from three to four hidden layers raises validation accuracy from 90.80% to 91.41%. Adding a fifth hidden layer reduces validation accuracy slightly to 91.20%. Thus, greater depth was not automatically beneficial in this experiment.

Architecture	Training Accuracy	Validation Accuracy
3-hidden	95.30%	90.80%
4-hidden	95.19%	91.41%
5-hidden	94.99%	91.20%

4.8 Comparison of Optimizers

Across the three architectures, SGD achieved the highest validation score among the tested optimizers. NAG and Momentum followed closely, while Adam and RMSProp performed somewhat lower. Batch Gradient Descent was substantially weaker, with a best validation accuracy of only 20.66%.

Optimizer	Best Validation Accuracy
SGD	91.41%
NAG	91.09%
Momentum	90.57%
Adam	90.41%
RMSProp	87.57%
Batch GD	20.66%

5. Conclusion

A fully connected neural network was successfully evaluated for five-class Fashion-MNIST image classification. The experiment compared three network depths with six optimization algorithms, resulting in 18 training configurations. The dataset contained 35,000 balanced images and was divided into 64% training, 16% validation, and 20% test data. The four-hidden-layer FCNN trained with SGD produced the highest validation accuracy of 91.41% and achieved 91.70% on the final test set.

Overall, the experiment demonstrates that a comparatively simple FCNN can classify the selected Fashion-MNIST categories effectively. The results show that increasing model depth beyond four hidden layers did not improve validation performance, while the choice of optimizer had a major impact on learning quality. Under the reported experimental settings, four hidden layers combined with SGD offered the strongest overall performance.